# STM32MP157C-DK2 Embedded Linux Platform Development

Heterogeneous Embedded System Design and Implementation

Sulaiman Khan

March 2026

# 1. Introduction

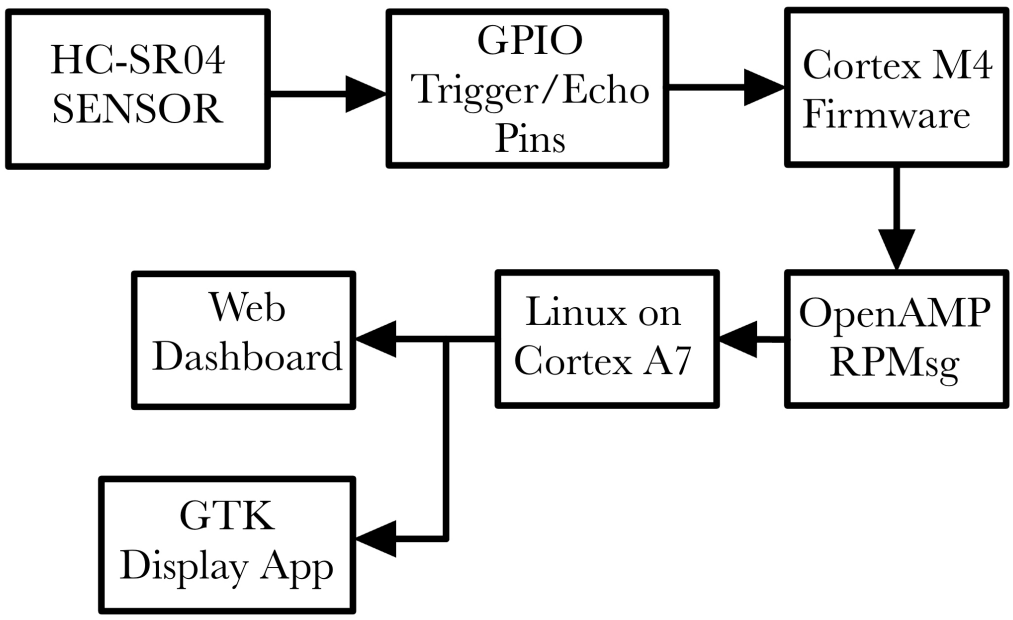
This project works with the STM32MP157C-DK2, using both processing cores of the board for their intended purposes, the Cortex-M4 handling real time sensor acquisition and the Cortex-A7 running Linux for application-level display and networking.

I used an HC-SR04 ultrasonic distance sensor interfaced with the Cortex-M4, which handles the precise timing required to generate trigger pulses and measure echo responses. Measured distance values are sent to the Linux side via OpenAMP RPMsg, an inter processor communication framework built on shared memory. On the Linux side, I first wanted to make a display interface of a web based dashboard over USB networking but I then thought it would make more sense to display it directly on the boards built in display so I developed a small GTK application to do it.
The project spans hardware design, real time firmware development, embedded Linux bring up, inter processor communication, and application level software serving as a demonstration of a modern heterogeneous embedded system.

Click here to watch demo video

# 2. System Architecture

The system is divided into multiple stages: sensor acquisition, real-time firmware processing, inter processor communication, and Linux-side display. I drew a block diagram shown below to show this system




## 2.1 HC-SR04 Sensor and GPIO Interface

The HC-SR04 ultrasonic sensor measures distance by emitting a sonic pulse and measuring the time it takes for the echo to return. The Cortex-M4 uses the sensor's Trigger pin with a 10 microsecond pulse, then measures the Echo pin's response. Distance is calculated as:

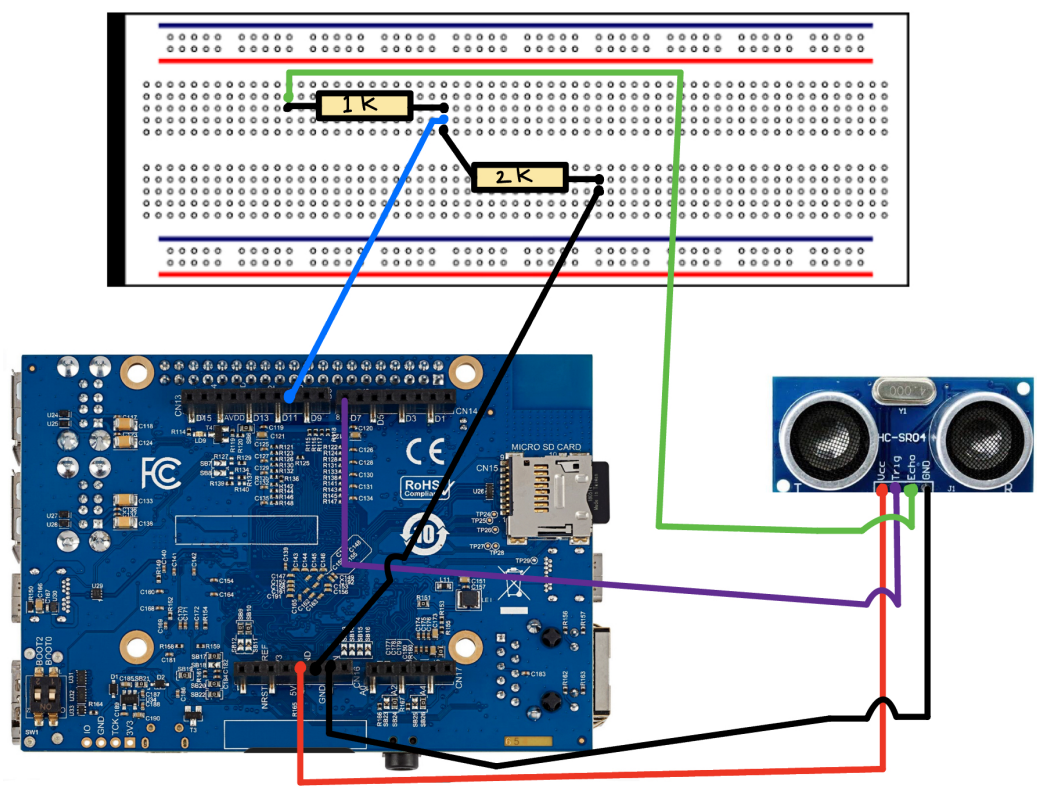
$$d = \frac{t \times 343}{2} \tag{1}$$

where $t$ is the echo pulse duration in seconds and 343 m/s is the speed of sound. The division by 2 accounts for the round-trip travel of the pulse.

A critical hardware consideration is that the HC-SR04 operates at 5V and its Echo pin outputs a 5V signal, while the board's GPIO pins are 3.3V logic. To safely interface the two, I made a simple voltage divider using a 1kΩ and 2kΩ resistor:

$$V_{out} = V_{in} \times \frac{R_2}{R_1 + R_2} = 5\ \text{V} \times \frac{2\ \text{k}\Omega}{1\ \text{k}\Omega + 2\ \text{k}\Omega} = 3.33\ \text{V} \tag{2}$$

This brings the Echo signal within the safe input range of the STM32 GPIO. I also drew a wiring diagram to better show exactly how the hardware was connected:




## 2.2 Cortex-M4 Firmware

The M4 firmware was written in C using STM32CubeIDE, starting from ST's OpenAMP TTY echo example as a base. The core logic reads the HC-SR04 sensor and continuously transmits distance values to the Linux side over RPMsg.

One challenge was achieving the microsecond-level timing the HC-SR04 requires. the trigger pulse must be exactly 10 microseconds wide and the echo pulse width must be measured precisely. Rather than configuring a hardware timer peripheral, I used the Cortex-M4's built in DWT cycle counter registers, which count CPU clock cycles directly and gave the resolution needed without extra peripheral setup.

## 2.3 OpenAMP and RPMsg

Before writing any sensor code, the first goal was simply proving that the M4 and A7 could talk to each other at all. OpenAMP is the framework ST provides for this. It uses a shared memory region to pass messages between the two cores entirely on chip, with no physical bus or extra hardware.

To test it, I loaded ST's echo example firmware onto the M4 and `"hello m4"` sent `/dev/ttyRPMSG0` from the Linux side. When the message came back echoed, it confirmed the full communication path was working meaning the M4 was running, and Linux could read and write to the M4. That was one of the most satisfying moments of the project, because everything that came after depended on that link working.

## 2.4 Linux-Side Display

Once distance readings were flowing through `/dev/ttyRPMSG0`, the next step was displaying them on the Linux side. The first approach was a Python Flask web dashboard running on the A7, which read the incoming RPMsg data and served a live scrolling graph accessible from a browser over USB networking. That worked, but it required an external device to view it.

Looking at the board, it has a built-in 4.3" touchscreen, and the idea of having the board display its own data without needing a laptop or phone felt like a much cleaner end result. That became my next goal.

The first attempt used Python and pygame to render a fullscreen display directly on the screen. That failed immediately. Pygame was not installed on the board and I didnt have the board connected to internet. I ended up using Python GTK, which was already installed on the board as part of the OpenSTLinux image. However, getting it running required a few extra steps. Weston, the Wayland compositor, needed to be running first, and the correct environment variables had to be set so GTK could find the display. Once those pieces were in place, the GTK app launched fullscreen on the touchscreen and displayed the live distance reading with color coding .

# 3. Software Design

## 3.1 M4 Firmware

The M4 firmware is based on ST's official OpenAMP TTY echo example from the STM32CubeMP1 firmware package, imported into STM32CubeIDE and modified to replace the echo logic with HC-SR04 sensor reading and distance transmission.

The main loop runs at 10Hz, reading the sensor every 100ms using `HAL_GetTick()` for interval tracking. Each reading is formatted as `DIST:123.45\n` and transmitted to Linux over the RPMsg virtual UART. If the echo pulse times out or the reading is out of range, `DIST:ERR\n` is sent instead. The firmware also listens for simple commands from Linux, sending `stop` over `/dev/ttyRPMSG0` pauses the sensor and `start` resumes it.

Timing was handled entirely through the DWT cycle counter. The counter is initialized at startup by enabling the trace unit and resetting `DWT_CYCCNT`. Two functions are built on top of it, `delay_us()` for blocking microsecond delays and `get_us()` for reading the current time.

## 3.3 GTK Display Application

The Linux-side display application is written in Python using GTK 3, running fullscreen under the Weston Wayland compositor on the board's 4.3" touchscreen.

A background thread continuously reads from `/dev/ttyRPMSG0`, parses lines beginning with `DIST:`, and stores the latest value in a shared variable protected by a threading lock.. The GTK main thread updates the display every 100ms using a `GLib` timeout, keeping the UI responsive without blocking on sensor reads.

The display shows the distance as a large centered number with a color that changes based on proximity, green for distances above 60cm, yellow between 20cm and 60cm, and red below 20cm. A progress bar at the bottom fills proportionally up to 200cm. While waiting for the first reading, the display shows a `WAITING FOR SENSOR` message.

# 4. Development Process and Challenges

The project definitely did not follow a straight path from idea to working system. A significant portion of the development time was spent not on the sensor application itself, but on getting the platform into a usable state, something that turned out to be the most valuable learning experience of the project.

The first major hurdle was establishing communication between the M4 and A7 cores. The board initially ran an older OpenSTLinux image from 2019 with kernel 4.19.9. After loading the M4 firmware and confirming through kernel logs that the RPMsg channels were being created, `/dev/ttyRPMSG0` never appeared. The driver existed, the channels existed, but I couldn't figure out why it wasn't working. After extensive debugging I figured that the old image was the root cause. Rather than trying to work with the old Linux, I reflashed the board entirely with a newer OpenSTLinux release using STM32CubeProgrammer via USB DFU mode. The reflash process introduced its own complications, But I used online resources to help me figure it out such as the ST YouTube channel which had a guide to doing this.

After successfully flashing the newer image and confirming kernel 6.6 was running, the RPMsg problem was still not immediately resolved. The new image did not include the OpenAMP M4 firmware by default, and the `rpmsg_tty` kernel module needed to be loaded manually with `modprobe`. Once the firmware was copied onto the board via USB flash drive and the M4 was started through remoteproc, `/dev/ttyRPMSG0` finally appeared. Sending `"hello m4"` through the device and receiving the echo back was one of the most satisfying moments of the entire project, it confirmed that the full A7 to M4 communication path was working end to end.

With the firmware finally working, I went through the challenges previously mentioned on getting the display to work but ended up with a clean final product. I learned a lot in this process because of all the problems I faced and how resourceful I had to be. With the platform bring up complete, I want to now develop a more practical and advanced application on the system